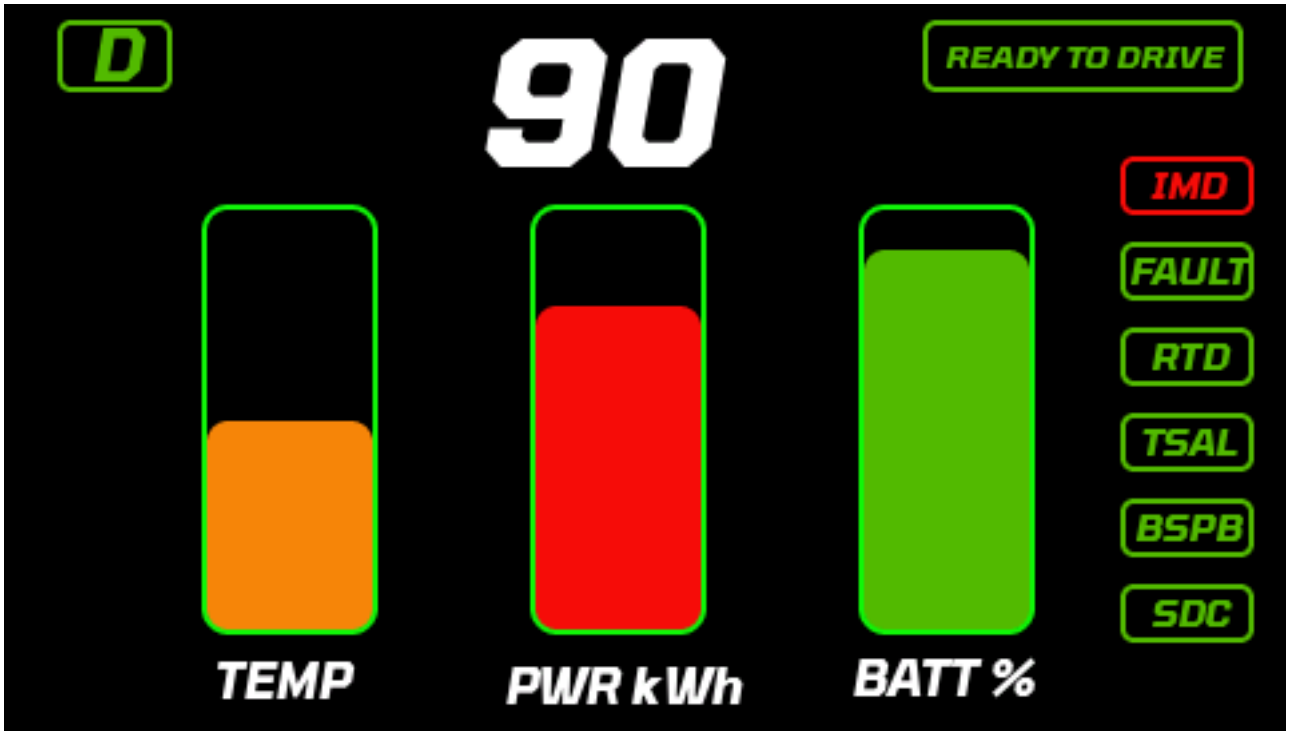
Demo and code review

Prepared 16 October 2026. Based on the current local working files on **develop**, including uncommitted changes. Review scope: maintained application logic, synthetic input pipeline, shared UI, logging and recorded playback. No source-code changes were made for this review.



Actual LVGL capture at 4.5 s: synthetic speed 90 km/h, mode D and an active IMD fault. These are pixels rendered by the shared C/LVGL implementation.

Verified run	Result
Simulation duration / steps	10 seconds / 1001 steps
CAN requests / replies	188 / 177
Validated exported records / captures	177 / 101

The desktop demo runs the production CAN request scheduler and decoder, RAM/eMMC loggers, portable UI bindings and LVGL. Hardware interfaces are replaced by memory implementations. The HTML player replays captured images; it does not execute live firmware.

Included deliverables: dashboard-demo.html is self-contained; user-inputs.json is the exact scenario used. Both files are also embedded as PDF attachments. The complete input JSON is printed in section 03.

Verification boundary: physical CAN compatibility, real FreeRTOS scheduling, board LCD/touch and power-loss storage behavior: **Insufficient data to verify.**

Open, run and change the demo

Open the supplied HTML

Open dashboard-demo.html in a normal browser. All 101 screen captures, capture-associated states, and downloadable scenario/snapshot/CSV data are embedded. The file does not require an accompanying image folder.

```
open "/Users/youseffarahat/Documents/hard-fs-dashboard/output/demo/dashboard-demo.html"
```

The player has play/pause, replay, speed, loop and time controls. Reduced-motion preferences start playback paused. The screen and displayed decoded state correspond to the same loaded capture. Playback selects recorded frames by timestamp and does not interpolate pixels.

Regenerate using the editable scenario

```
cd "/Users/youseffarahat/Documents/hard-fs-dashboard"

.venv/bin/python -m tools.dashboard_sim \
  --scenario tools/can_sim/scenarios/user-inputs.json \
  --open
```

The command builds the desktop C host, runs the scenario, writes build/dashboard-sim/, then asks the operating system to open index.html. Editing JSON requires a new run; the browser controls only playback. Use --output build/my-test to retain another run. [S1, S2]

First-time environment setup

```
python3 -m venv .venv
.venv/bin/python -m pip install \
  -r tools/can_sim/requirements.txt \
  -r tools/requirements-dev.txt
```

A host Clang/GCC compiler with address/undefined-behavior sanitizer support and CMake are required. The existing environment successfully built and ran the host in this chat. Node is needed for the playback regression tests, not for playing the generated HTML.

Option	Effect
--step-ms 10	Default deterministic simulation step; allowed range 1..1000 ms.
--capture-every-ms 100	Default capture interval: 10 frames/s at the default step. Set 50 for 20 frames/s; 0 disables capture.
--duration 10	Override scenario duration in seconds.
--scenario .../dashboard.json	Use the supplied 20-second stress/fault scenario.

Generated outputs: scenario.json, summary.json, snapshots.jsonl, log.bin, log.csv, image captures and index.html. Default runs use a multi-file viewer; the supplied standalone HTML embeds its dependencies. [S1, S2]

Complete user input

This is the exact JSON used for the verified ten-second run. The separate user-inputs.json file preserves the original bytes. Times t, start and end are elapsed simulation seconds. [A1]


```
{
  "version": 1,
  "duration": 10,
  "signals": {
    "pack_voltage_v": 350,
    "pack_summed_voltage_v": 350.12,
    "pack_current_a": {"interpolation": "linear", "points": [
      {"t": 0, "value": 0}, {"t": 4, "value": 80},
      {"t": 7, "value": -20}, {"t": 10, "value": 0}
    ]},
    "pack_soc_pct": 90,
    "highest_temperature_c": 45,
    "lowest_temperature_c": 20,
    "average_temperature_c": 30,
    "lowest_cell_voltage_v": 3.4,
    "highest_cell_voltage_v": 4.1,
    "average_cell_voltage_v": 3.7
  },
  "auxiliary": {
    "speed_kph": {"interpolation": "linear", "points": [
      {"t": 0, "value": 0}, {"t": 5, "value": 100}, {"t": 10, "value": 0}
    ]},
    "drive_mode": {"points": [
      {"t": 0, "value": "N"}, {"t": 1, "value": "D"}, {"t": 9, "value": "R"}
    ]},
    "statuses": {
      "imd": {"points": [
        {"t": 0, "value": "ok"}, {"t": 4, "value": "fault"}, {"t": 6, "value": "ok"}
      ]},
      "fault": "ok", "rtd": "ok", "tsal": "ok",
      "bspd": "ok", "sdc": "ok", "ready_to_drive": "ok"
    }
  },
  "faults": [{"type": "pause", "start": 7, "end": 8}]
}
```


The CAN pause applies from 7.000 s inclusive to 8.000 s exclusive. Auxiliary speed, mode and status timelines continue independently. The scenario is a prescribed test sequence, not a battery or vehicle dynamics model.

Input meaning and data sources

Information	Desktop source	Board firmware source
Pack voltage, summed voltage, current and SOC	JSON values encoded into virtual CAN replies	Accepted FDCAN1 responses
Cell-voltage and temperature high/low/average	Separate JSON signals encoded into replies	Separate received PID measurements
Speed / N-D-R mode	Explicit auxiliary JSON timelines	Automatic placeholder generator
Seven digital status indicators	Explicit unknown / ok / fault timelines	Placeholder generator marks all healthy
Current and power statistics	Calculated in production C	Same production C algorithms
RAM / eMMC usage	Counters over memory replacements	Counters over board storage

Received versus calculated: SOC is received, not estimated. The high/low/average cell voltages and temperatures are separate received values, not summaries calculated from individual cells. Summed pack voltage is also a received field. Current and power statistics are calculated locally. [S3, S4]

How timeline values are chosen

A scalar is constant. A points array defaults to step interpolation: the last point at or before the requested time applies. Numeric linear tracks use the fraction (t - t0) / (t1 - t0) between adjacent points. Before the first or after the last point, the endpoint value is retained. Point times must be nonnegative and strictly increasing. [S5]

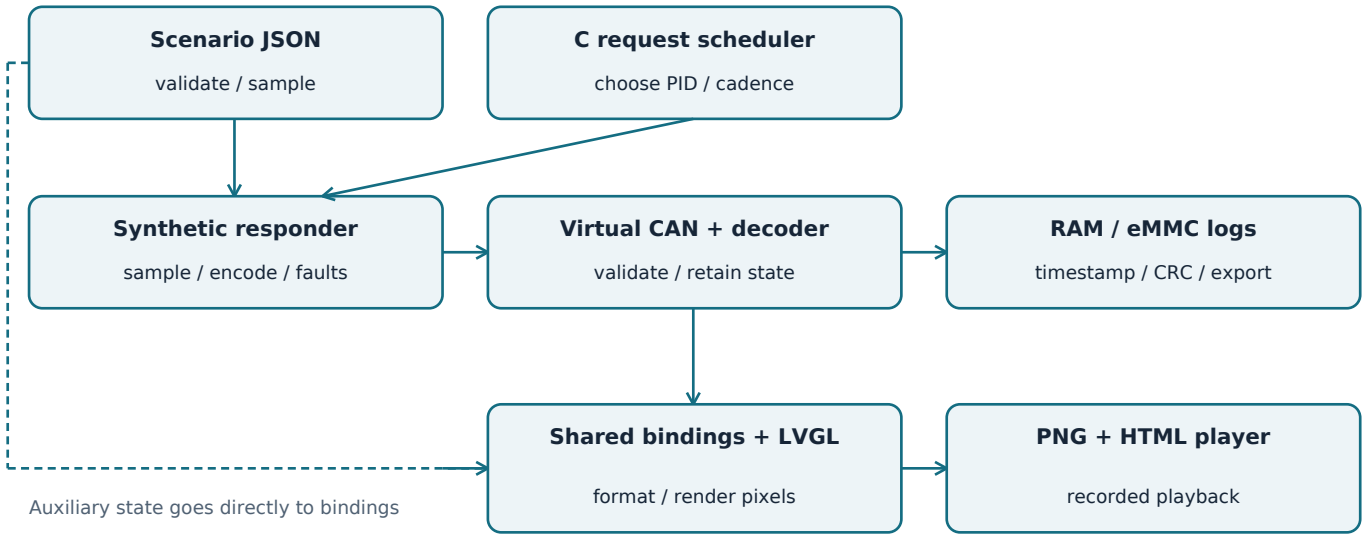
The validator rejects unknown fields, duplicate JSON keys, booleans in numeric fields, nonfinite numbers, unsupported interpolation, unsupported modes/statuses and values outside representation ranges. Omitted battery signals become synthetic zero. Omitted auxiliary inputs become unknown. Normal range validation is not a vehicle safety specification. [S5, S6]

What this particular scenario asks for

Pack voltage stays at 350 V, summed voltage at 350.12 V and SOC at 90%. Current ramps 0 to 80 A by 4 s, to -20 A by 7 s, then to 0 A by 10 s. Speed ramps 0 to 100 km/h by 5 s, then back to 0 by 10 s. Mode changes N to D at 1 s and R at 9 s. IMD changes to fault at 4 s and recovers at 6 s; the other statuses stay healthy. Replies to new requests are suppressed from 7 to 8 s. [A1]

These tracks are independent. A speed increase does not cause a current increase in the code. CAN sampling and quantization also mean that decoded values need not equal the scenario value at the exact capture time.

The processing pipeline, step by step



1. Parse: Python reads exact scenario bytes and validates them before running. The source input is protected against artifact overwrite. The host adapter also validates numeric ranges, command field counts, payload syntax and permitted frame sizes. [S1, S2, S5, S7]

2. Advance time: the runner supplies absolute milliseconds to a persistent C process. At the default 10 ms step, a 10-second run includes t=0 and t=10000, giving 1001 steps. It does not sleep for each simulated interval. [S2]

3. Request and respond: production C selects the PID and request timing. Python sends those bytes unchanged through a same-process virtual bus. The responder samples the requested signal and schedules its encoded reply. Current-tick requests enter C as replies on a subsequent simulation step. [S2, S3, S6, S8]

4. Decode and retain: production C accepts or rejects each reply. Acceptance changes the matching field in a copy of the retained snapshot. Other fields keep their prior values. A poll can accept multiple frames and publish one combined updated snapshot. [S3]

5. Record and draw: accepted updates are recorded in RAM and replacement eMMC. Battery and explicit auxiliary values are applied to shared widgets. LVGL advances by elapsed simulated time and renders actual pixels, which are saved as RGB captures and PNGs. [S2, S7, S9]

6. Export and replay: production C emits the binary log. Python validates its format and CRCs, writes CSV, and packages capture timestamps/states. JavaScript selects the capture at or before playback time and keeps the loaded image paired with its state. [S2, S10, S11]

CAN protocol, conversion and timing

Implemented requests use standard classic-CAN ID 0x7E3 and service 0x22. Replies use ID 0x7EB and service 0x62. This is the source-defined contract, not verified compatibility with the eventual BMS. [S3, S6]

PID	Measurement	Raw conversion
F00D	Pack voltage	Unsigned 16-bit big-endian; 0.1 V/count.
F014	Summed pack voltage	0.01 V/count; stored as decivolts using (raw + 5) / 10.
F015	Pack current	(raw - 32767) deciamps; upper extreme saturates to int16.
F00F	SOC	(raw byte + 1) / 2 percent; capped at 100.
F028 / F029 / F02A	High / low / average temperature	Signed 8-bit Celsius.
F032 / F033 / F034	Low / high / average cell voltage	Unsigned 16-bit big-endian; 100 microvolts/count.

Byte-level example: pack voltage of 350 V

```
Request ID:  0x7E3
Payload:     03 22 F0 0D 00 00 00 00

Response ID: 0x7EB
Payload:     05 62 F0 0D 0D AC 00 00

0x0DAC = 3500 counts; 3500 x 0.1 V = 350 V
```

Response acceptance requires standard ID, classic data-frame metadata, exactly eight payload bytes, service 0x62, a known PID and its exact expected length/PCI byte. Invalid replies preserve the retained state. Multi-frame ISO-TP reassembly is absent. [S3]

Request schedule and measurement age

Only one PID is pending at a time. When idle, the scheduler sends after at least 50 ms since the last request. A pending request expires only when elapsed time is greater than 100 ms, then the sequence advances. A matching accepted reply also advances it. With ten PIDs and prompt replies, a nominal round takes about 500 ms; losses and delays extend it. [S3]

The state therefore mixes measurement times. Unsigned tick subtraction handles rollover, but there are no per-field timestamps or stale markers. Valid unsolicited/duplicate replies update the state too; a matching pending PID is not required for acceptance. [S3]

Synthetic encoding rounds to the nearest count with half values away from zero. Representation bounds include 0..6553.5 V pack voltage, 0..655.35 V summed voltage, -3276.7..3276.7 A current, -128..127 C temperature and 0..6.5535 V cell voltage. These are encoding limits, not operating limits. [S6]

Calculations and interface behavior

What is calculated locally

Every accepted current reply contributes to persistent minimum, maximum, signed sum and sample count. Mean rounding is performed in integer units. Duplicate accepted currents count as additional samples. These statistics have no sliding window, time weighting or public reset API. [S3]

```
P_kW = V_volts * I_amps / 1000

Using stored deci-units:
P_kW = round(I_deciamps * V_decivolts / 100000)
```

Power is sampled only on current replies. It uses nonzero retained pack voltage, or summed voltage as fallback, without a freshness check. Each power sample is rounded to whole kW before power min/max/mean accumulation. There is no integration of power over time into kWh. [S3]

Battery widgets and debug labels

Main-screen bars receive highest temperature, rounded absolute current in amps, and SOC. Temperature and current are clamped to 1..100 before drawing; generated bar logic also clamps to 1..100. Thus a zero input still draws at the minimum. The current bar loses sign and clips above 100 A. [S4, S12]

Debug labels show signed whole Celsius, cell voltages rounded to millivolts, signed whole-kW power statistics and logger occupancy percentages. Despite their generated Speed names, two debug handles show RAM and eMMC usage; those names do not establish measurement meaning. [S4]

Known units defect: the center bar caption says PWR kWh, but its input is current magnitude in amps. It is neither instantaneous power nor energy. [S4, S12]

Auxiliary validity and status visibility

Known speed is rounded to whole km/h. Unknown speed or mode displays a dash. Mode accepts N, D and R. Each status carries validity and a fault flag: unknown is gray, valid healthy is green, and valid fault is red. [S4, S7]

All seven status indicators are visible for the first 3000 ms. Afterward, all disappear only when every status is known and healthy. Any unknown/faulted status makes all seven visible, with their individual colors. Names are IMD, Fault, RTD, TSAL, BSPD, SDC and Ready to Drive; no physical polarity or vehicle bit encoding follows from these names. [S4]

Temperature/current bar colors are green through 40, orange through 65 and red above 65. SOC reverses the low/high colors. These fixed presentation thresholds are not validated vehicle fault thresholds. [S12]

Preserved board-only auxiliary demonstration

The firmware GUI calls a generator that moves speed by 3 per qualifying 200 ms update, reverses at 0 and 160, and cycles N/D/R every 5000 ms. It marks every digital status known and healthy. Host scenarios bypass that generator and supply explicit auxiliary state. [S4, S13]

Board startup and task execution

Firmware startup and the host simulation are separate entry points. The host links shared algorithms but excludes the STM32 reset path, interrupts and FreeRTOS tasks. [S14, S15]

Startup sequence

The board reset/runtime path reaches main. main configures MPU regions before enabling instruction/data caches; initializes HAL, system clocks and COM1; initializes QSPI and enables memory mapping; creates application tasks; then starts the FreeRTOS scheduler. Fatal initialization errors enter an interrupt-disabled infinite loop. [S14]

The firmware build emits internal and QSPI images. The documented flash script programs QSPI at 0x90000000 before internal flash at 0x08000000 and resets the board. No flashing was performed for this review. [S15, S16]

Task	Timing / ownership	What it does
CAN acquisition	10 ms polling; higher priority than GUI/console	Initialize/retry CAN, decode updates, timestamp, log to RAM/eMMC, overwrite latest-state queue.
GUI	Nominal 5 ms service; owns normal LVGL mutations	Initialize LVGL/display/touch/screens, consume queued battery state, generate auxiliary placeholders, service LVGL.
UART console	One-byte reads; 50 ms UART timeout	Dispatch inspection, reset, flush and export commands; synchronous dumps can delay commands.

The queue holds one full snapshot. Overwrite preserves the latest state while allowing intermediate display states to be skipped. Acquisition logs before queue overwrite. Several accepted frames in one poll yield one combined record. eMMC append and queue return values are ignored in the board acquisition task. [S13]

Physical display, touch and CAN boundaries

The LCD port initializes SDRAM and registers two full-screen framebuffers with LVGL. Flush performs cache maintenance and switches LTDC to the rendered buffer. Touch reads BSP coordinates into LVGL pointer events; initialization/read failures are not consistently surfaced. Neither port runs in the desktop demo. [S17, S18]

CAN configuration selects FDCAN1, PH14 RX, PH13 TX and AF9. Initialization selects normal transmit-capable classic CAN with automatic retransmission. From the configured 25 MHz HSE and timing fields, the nominal bitrate computes to 500 kbit/s. This is a configuration calculation, not a physical measurement or proof of network compatibility. [S3, S19]

The source also documents unresolved board memory/panel assumptions. This report does not independently revalidate CAD, fitted-board identity, electrical routing or actual panel timing. Physical hardware operation: **Insufficient data to verify.**

Logging, persistence and export

RAM history

The RAM logger stores 4096 timestamped full battery snapshots. The next-write index advances modulo capacity; after filling, new writes replace the oldest. A sticky wrapped flag records that overwriting occurred. Short critical sections protect each operation. A multi-row CSV export is not one frozen transaction because rows are copied individually while acquisition may continue. [S20]

eMMC storage

The logger requests 262144 blocks of 512 bytes, beginning at block 4096 (2 MiB), limited by reported card capacity. One block holds session metadata; following blocks hold native timestamp/snapshot/CRC records. This is raw block storage without filesystem allocation coordination. [S10]

Appending copies a zero-initialized native record into a staged block and increments the logical record count. A successful append may leave the sample only in RAM. A partial flush retains the written tail so later appends can fill and rewrite that same block without inserting false records. Dirty data/metadata state supports retries after write failures. [S10]

Initialization reads recognized old metadata to choose the next session ID, then starts a new empty logical session. It does not resume the previous records, erase all old data, or provide a power-failure journal/recovery mechanism. The board task proceeds even if eMMC initialization fails and ignores append failure. [S10, S13]

Portable export and validation

Binary export emits a 24-byte HFDB header, portable 42-byte little-endian rows and a 16-byte HFDE footer. The reader checks framing, counts and checksums. The demo additionally requires stored-record and dump-record CRC validity before writing CSV. CRC success establishes integrity under these checks, not truth of the sensor measurements. [S2, S10, S11]

The binary/CSV schema contains timestamp, battery measurements/statistics and flags. Speed, mode and auxiliary statuses are recorded separately in snapshots.jsonl. A poll with multiple accepted replies logs a combined snapshot, rather than one record per reply or GUI tick. [S2, S7, S13]

```
.venv/bin/python dump_emmc_bin.py \
  --input-bin build/dashboard-sim/log.bin \
  --output-prefix build/redecoded
```

Console command	Action
h / ?; i; d; c	Help; RAM info; RAM CSV dump; clear RAM history.
e; f; x	eMMC info; flush; start new empty eMMC session.
p; b; v	eMMC CSV; binary export; toggle CAN diagnostics.

Exports run synchronously; eMMC export holds its mutex while output is sent. It can delay acquisition logging. Temporarily disabling CAN diagnostics does not establish exclusive ownership of every UART writer. The host uses memory storage/UART substitutes and a 64-block test region, not the physical card capacity. [S7, S10, S13]

Review findings and verified behavior

Finding	Consequence
No battery validity/age metadata	Startup zero and retained old values can be displayed as measurements. A timeout does not mark an individual value stale.
Auxiliary board values are placeholders	Displayed speed, mode and healthy statuses do not establish actual vehicle conditions. fault_flags is not populated by the ten-PID decoder.
PWR kWh is fed current magnitude	Wrong quantity and units; sign is lost; high values clip.
Sample-based statistics / asynchronous fields	Duplicates affect means; current can use an older voltage; no time weighting or energy integration.
Staged writes / ignored board append failures	A visible update or increased logical count does not prove durable persistence.
New session on eMMC initialization	Previous logical records are not automatically resumed after reboot.

Observed timeline from the generated run

Time	Observed screen / state
0 s	Speed 0, mode N; status group visible during boot.
3 s	Speed 60, mode D; all statuses healthy and hidden.
4.5 s	Speed 90, mode D; IMD fault makes the group visible.
6 s	Speed 80; IMD healthy again and group hidden.
7 to <8 s	No accepted battery updates; retained current stays -8.3 A while speed continues changing.
10 s	Speed 0, mode R; 177 battery log records.

These observations were extracted from the 1001 saved snapshots. At 10 s, retained current is -1.7 A although the prescribed track reaches 0 A: the latest current response predates the final capture. Input values, acquisition time and display time must be distinguished. [A1, A2]

Fault injection scope

The responder supports pause, drop, delay, wrong ID/service/PCI, short DLC, unknown PID, duplicates, bursts and raw-value overrides. The supplied user scenario exercises a pause; the separate dashboard.json stress scenario includes additional cases. Virtual traffic does not model electrical faults, arbitration, real controller FIFO timing or transceiver behavior. [S8, S21]

Verification and evidence boundary

Check executed in this chat	Result
Desktop host build and ten-second scenario	Successful; 188 requests, 177 replies, 177 validated exported rows and 101 captures.
Dashboard / CLI / playback Python cases	12 passed; includes Node-backed playback regression checks.
Virtual CAN to production decoder integration	4 passed; ten signals, malformed frames, retained state and statistics.
Storage / binary export regression cases	17 passed; partial blocks, corruption, capacity, reset and retry behavior.
C UI-binding test	1 passed.

The focused Python total is 33 cases, plus the C UI-binding test. Host C compilation uses address and undefined-behavior sanitizers for supported Clang/GNU compilers. Passing focused tests is not a claim of exhaustive verification. The STM32 firmware image was not rebuilt or flashed during this review. [S15, S22-S25]

Commands used for the focused checks

```
.venv/bin/python -m unittest discover -s tests \
  -p 'test_dashboard*.py' -v
.venv/bin/python -m unittest discover -s tests \
  -p 'test_can_pipeline.py' -v
.venv/bin/python -m unittest discover -s tests \
  -p 'test_storage_export.py' -v
.venv/bin/ctest --test-dir build/dashboard-host \
  --output-on-failure
```

Additional packaging checks for these deliverables

Every standalone HTML capture was decoded from its embedded data URI and compared byte-for-byte with the original PNG. Capture states were preserved. All HTML src/href resources were checked to be internal data or fragment links. The supplied JSON matches both the run scenario and the editable source scenario byte-for-byte. The PDF includes visible JSON plus attached copies of the HTML and JSON.

What remains unverified

The host does not execute MCU startup, real FreeRTOS scheduling, controller registers, electrical CAN or actual touch/LCD/storage devices. It does not establish bitrate compatibility, wiring correctness, fitted hardware identity, DMA/cache behavior under hardware load, concurrent task behavior, or power-loss recovery. For these: **Insufficient data to verify.**

The in-app browser automation tool rejected the local file URL. The original rendered images were inspected directly; recorded playback behavior was checked by the regression tests. Interactive browser playback was not visually verified through that tool. The ambient browser state later reported the local player URL, which by itself does not establish successful control interaction.

Source map and artifact provenance

Repository root: /Users/youseffarahat/Documents/hard-fs-dashboard. Paths below are relative to this root; line numbers are navigation anchors from the reviewed working files. These references identify implementation evidence rather than an audit of every vendor routine.

S1 tools/dashboard_sim/__main__.py:17
CLI and browser launch

S2 tools/dashboard_sim/runner.py:227
Clock, closed loop, artifacts, CRC checks

S3 Src/app_can.c:269,539,774,993
Power, statistics, decoder, scheduler

S4 Src/app_dashboard_ui.c:258,318,390
Placeholder generator and UI bindings

S5 tools/can_sim/scenario.py:295
Scenario parsing and sampling

S6 tools/can_sim/protocol.py:39,88
Signal table and response encoding

S7 tools/dashboard_host/main.c:215
Validated command / state / logging boundary

S8 tools/can_sim/engine.py:34
Responder and timed faults

S9 tools/dashboard_host/render.c:59,92
Memory display and captured pixels

S10 Src/app_emmc_log.c:473,537,768
Sessions, append and binary dump

S11 dump_emmc_bin.py
Portable dump reader / CSV

S12 ui/screens/ui_MainScreen.c:49
Bar drawing and screen events

S13 Src/app_tasks.c:238,289,374
GUI, acquisition, console, task creation

S14 Src/main.c:41
Board initialization

S15 CMakeLists.txt; tools/dashboard_host/CMakeLists.txt
Separate firmware and host builds

S16 .vscode/flash_board.sh:102
QSPI then internal programming

S17 Src/Lvgl_Porting/lvgl_port_lcd.c:221,244
LTDC flush and framebuffer setup

S18 Src/Lvgl_Porting/lvgl_port_touchpad.c
BSP touch input

S19 Src/app_can_config.c
FDCAN1 / PH13 / PH14 configuration

S20 Src/app_data_log.c:79
4096-record RAM ring

S21 tools/can_sim/scenarios/dashboard.json
Separate stress scenario

S22 tests/test_dashboard_cli.py; test_dashboard_playback.py
CLI and playback checks (tests/)

S23 tests/test_dashboard_sim.py
Scenario / render / export integration

S24 tests/test_can_pipeline.py; test_storage_export.py
CAN and storage checks (tests/)

S25 tests/dashboard_ui/test_binding.c
C UI-binding assertions

A1 build/dashboard-review-20261016/scenario.json
Exact input scenario

A2 build/dashboard-review-20261016/snapshots.jsonl
Saved per-step state

A3 build/dashboard-review-20261016/summary.json
Run counts and declared scope